

# Description-to-Primitive Mapping Analysis

## Overview

This document analyzes how natural language descriptions map to primitives in the card game rule learning system, with particular focus on: 1. Current high-level descriptions and their primitive mappings 2. What happens when higher-level primitives are dropped 3. Whether standardized “primitive vocabulary” is needed (like Wong et al. 2021) 4. Concrete examples of compositional descriptions

---

## Part 1: Current Description → Primitive Mappings

### 1.1 The Current Situation

Each rule in `catalogue.py` has: - **name**: Short human-readable name - **description**: More detailed explanation - **primitives\_used**: List of primitives in the program

Here’s how current descriptions map to primitives:

| Rule ID           | Description                                         | Primitives Used                             | Key Mapping                                         |
|-------------------|-----------------------------------------------------|---------------------------------------------|-----------------------------------------------------|
| Uniform_color     | “All cards are either black ( / ) or red ( / ).”    | uniform, get_color, unique_count, eq        | “same color” → uniform(get_color)                   |
| Has_pair_ranks    | “Some two cards share the same rank, e.g., two 9s.” | unique_count, get_rank, length, lt          | “pair” → lt(unique_count(get_rank), length)         |
| Sorted_by_rank    | “Read ranks left-to-right; they never go down”      | is_sorted, map, get_rank_val                | “sorted/never go down” → is_sorted                  |
| Exactly_two_suits | “Exactly two distinct suits”                        | unique_count, get_suit, eq                  | “exactly two suits” → eq(unique_count(get_suit), 2) |
| Ends_same_color   | “First and last cards are both red or both black”   | terminals_equal, get_color, first, last, eq | “first and last same” → terminals_equal             |

### 1.2 Description Word → Primitive Associations

From the 45 rules, we can extract these word-primitive associations:

**Property Words:** | Word/Phrase | Associated Primitives | | “same”, “uniform”, “all the same” | uniform, eq, unique\_count | | “pair”, “two share” | lt, unique\_count, length | | “sorted”, “increasing”, “non-decreasing” | is\_sorted, lte | | “first”, “last” | first, last, head | | “half”, “halves”, “split” | halves, left\_half, right\_half | | “color”, “red”, “black” | get\_color | | “suit”, “hearts”, “spades”, etc. | get\_suit, specific suit constants | | “rank”, “number” | get\_rank, get\_rank\_val | | “exactly N” | eq, count | | “at least”, “at most” | gte, lte | | “adjacent”, “neighbors” | adjacent\_pairs, shifted\_pairs | | “palindrome”, “forward/back” | seq\_palindrome, reverse, eq |

---

## Part 2: Impact of Dropping Higher-Level Primitives

### 2.1 Primitives You Want to Consider Dropping

You mentioned: - first\_half, second\_half - has\_suit, has\_color - count\_suit, count\_color - all\_same\_suit, all\_same\_color - n\_unique\_suits, n\_unique\_ranks, n\_unique\_colors

## 2.2 What These Primitives Compute

| Higher-Level Primitive       | What It Does                  | Lower-Level Decomposition                        |
|------------------------------|-------------------------------|--------------------------------------------------|
| <code>all_same_color</code>  | Check if all cards same color | <code>eq(unique_count(map(get_color)), 1)</code> |
| <code>all_same_suit</code>   | Check if all cards same suit  | <code>eq(unique_count(map(get_suit)), 1)</code>  |
| <code>n_unique_colors</code> | Count distinct colors         | <code>unique_count(map(get_color))</code>        |
| <code>n_unique_suits</code>  | Count distinct suits          | <code>unique_count(map(get_suit))</code>         |
| <code>count_suit(s)</code>   | Count cards of suit s         | <code>length(filter(eq(get_suit, s)))</code>     |
| <code>count_color(c)</code>  | Count cards of color c        | <code>length(filter(eq(get_color, c)))</code>    |
| <code>has_suit(s)</code>     | Check if any card has suit s  | <code>any(eq(get_suit, s))</code>                |
| <code>has_color(c)</code>    | Check if any card has color c | <code>any(eq(get_color, c))</code>               |
| <code>first_half</code>      | Get left half of hand         | <code>take(div(length, 2))</code>                |
| <code>second_half</code>     | Get right half of hand        | <code>drop(div(length, 2))</code>                |

## 2.3 Program Complexity With vs. Without Higher-Level Primitives

### Example 1: “All cards have the same color” (`Uniform_color`)

WITH higher-level primitives:

```
h. all_same_color(h)
Program length: 2 tokens
```

WITHOUT higher-level primitives:

```
h. eq(unique_count(map(get_color, h)), 1)
Program length: 6 tokens
```

### Example 2: “Both halves have a heart or neither does” (`Halves_hearts_presence_equal`)

WITH higher-level primitives:

```
h. eq(has_suit(HEARTS, first_half(h)), has_suit(HEARTS, second_half(h)))
Program length: 8 tokens
```

WITHOUT higher-level primitives:

```
h. eq(any(eq(get_suit, HEARTS), take(div(length(h), 2), h)),
      any(eq(get_suit, HEARTS), drop(div(length(h), 2), h)))
Program length: 16 tokens
```

### Example 3: “Exactly two suits appear” (`Exactly_two_suits`)

WITH higher-level primitives:

```
h. eq(n_unique_suits(h), 2)
Program length: 3 tokens
```

WITHOUT higher-level primitives:

```
h. eq(unique_count(map(get_suit, h)), 2)
Program length: 5 tokens
```

## 2.4 Search Space Implications

| Primitive Level      | Avg Program Length | Estimated Search Multiplier   |
|----------------------|--------------------|-------------------------------|
| With higher-level    | ~4 tokens          | 1× (baseline)                 |
| Without higher-level | ~8 tokens          | ~1000× (exponential in depth) |

**Key insight:** Removing higher-level primitives makes programs ~2× longer, which translates to ~1000× harder search (since search is exponential in program depth).

## Part 3: Standardized Primitive Vocabulary

### 3.1 Wong et al. (2021) Approach

In LAPS, descriptions use a **standardized vocabulary** that maps directly to program operations:

"large six gon" → `shape.size(shape) = LARGE sides(shape) = 6`

Each word has a fixed meaning: - "large" → size predicate with LARGE constant - "six" → numerical constant 6 - "gon" → polygon/sides predicate

### 3.2 Do We Need This?

**Arguments FOR standardized vocabulary:** 1. **Precise mapping:** Each word has exactly one meaning 2. **Compositional:** Complex descriptions compose predictably 3. **Learnable:** Simpler for translation model to learn 4. **No ambiguity:** "pair" always means the same thing

**Arguments AGAINST:** 1. **Unnatural:** "all red-or-black uniform" vs "all cards same color" 2. **Cognitive cost:** Participants must learn vocabulary 3. **Limited transfer:** Doesn't generalize to novel descriptions

### 3.3 Proposed Standardized Vocabulary for Card Games

If we adopt a standardized vocabulary, here's a proposal:

**Property Extractors:** | Standardized Term | Primitive | Example Usage | |  
 —| | SUIT | `get_suit` | "SUIT of first" | | RANK | `get_rank` | "RANK equals" | | RANK-VAL | `get_rank_val` |  
 | "RANK-VAL sum" | | COLOR | `get_color` | "COLOR uniform" | | PARITY | `get_parity` | "PARITY same" |

**Aggregation Operators:** | Standardized Term | Primitive | Example Usage | |  
 —| | UNIFORM | `uniform / unique_count == 1` | "SUIT UNIFORM" | | COUNT | `length, count` |  
 "COUNT equals 5" | | UNIQUE-COUNT | `unique_count` | "SUIT UNIQUE-COUNT equals 2" | | ANY | `any` |  
 | "ANY SUIT equals hearts" | | ALL | `all` | "ALL RANK-VAL less 10" | | SUM | `sum` | "RANK-VAL SUM  
 exceeds 30" | | MAX | `max` | "RANK-VAL MAX" |

**Position Operators:** | Standardized Term | Primitive | Example Usage | |  
 —| | FIRST | `head, first` | "FIRST SUIT" | | LAST | `last` | "LAST COLOR" | | LEFT-HALF | `first_half,`  
`take(n/2)` | "LEFT-HALF SUIT UNIFORM" | | RIGHT-HALF | `second_half, drop(n/2)` | "RIGHT-HALF  
 COLOR" | | ADJACENT | `adjacent_pairs` | "ADJACENT SUIT equals" |

**Comparators:** | Standardized Term | Primitive | Example Usage | |  
 —| | EQUALS | `eq` | "SUIT EQUALS hearts" | | EXCEEDS | `gt` | "COUNT EXCEEDS 3" | | LESS | `lt` | "RANK-VAL  
 LESS 5" | | AT-LEAST | `gte` | "COUNT AT-LEAST 2" | | AT-MOST | `lte` | "RANK-VAL AT-MOST 10" |

**Combinators:** | Standardized Term | Primitive | Example Usage | |  
 —| | AND | `and` | "UNIFORM COLOR AND SORTED" | | OR | `or` | "SUIT EQUALS hearts OR diamonds" | |  
 NOT | `not` | "NOT ANY ACE" | | BOTH-OR-NEITHER | `eq(P1, P2)` | "LEFT-HALF RIGHT-HALF BOTH-  
 OR-NEITHER UNIFORM" |

## Part 4: Concrete Description Examples

### 4.1 Examples WITH Higher-Level Primitives

#### Rule: Uniform\_color

Natural: "All cards are the same color (all black or all red)"

Standardized: "COLOR UNIFORM"

Program: `h. all_same_color(h)`

#### Rule: Has\_pair\_ranks

Natural: "At least one pair (two cards with same rank)"

Standardized: "RANK UNIQUE-COUNT LESS COUNT"

Program: `h. lt(unique_count(get_rank)(h), length(h))`

#### Rule: Halves\_uniform\_color\_equal

Natural: "Both halves are uniform in color (or both not)"

Standardized: "LEFT-HALF RIGHT-HALF BOTH-OR-NEITHER COLOR UNIFORM"

Program: `h. eq(uniform(get_color, left_half(h)), uniform(get_color, right_half(h)))`

### 4.2 Examples WITHOUT Higher-Level Primitives

#### Rule: Uniform\_color (decomposed)

Natural: "All cards are the same color"

Standardized: "COLOR UNIQUE-COUNT EQUALS one"

Program (without `all_same_color`):

`h. eq(unique_count(map(get_color, h)), 1)`

#### Rule: Halves\_hearts\_presence\_equal (decomposed)

Natural: "Both halves either have a heart or neither does"

Standardized: "LEFT-HALF RIGHT-HALF BOTH-OR-NEITHER ANY SUIT EQUALS hearts"

Program (without `has_suit`, `first_half`, `second_half`):

`h. eq(any(c. eq(get_suit(c), HEARTS), take(div(length(h), 2), h)),  
any(c. eq(get_suit(c), HEARTS), drop(div(length(h), 2), h)))`

### 4.3 Language Can Replace Higher-Level Primitives IF...

Language descriptions CAN compensate for missing higher-level primitives **if**:

1. **Recognition network learns the mapping:**
  - "all same color" → boost `unique_count`, `eq`, `get_color`, 1
  - "has a heart" → boost `any`, `eq`, `get_suit`, `HEARTS`
2. **Phrases consistently trigger correct primitives:**
  - "first half" → boost `take`, `div`, `length`
  - "both halves same" → boost `eq` + the half primitives
3. **Compositional structure is preserved:**
  - "left half has hearts" decomposes into:
    - "left half" → `take(div(length, 2))`
    - "has" → `any`
    - "hearts" → `eq(get_suit, HEARTS)`

**However**, there's a crucial asymmetry: - Higher-level primitives: 1 token - Decomposed version: 3-8 tokens  
- **More tokens = exponentially harder search**

Language can help the recognition model **prioritize the right primitives**, but it cannot change the fundamental search complexity.

#### 4.4 Recommendation: Hybrid Approach

**Keep some higher-level primitives for common patterns:** - `uniform(F)` - too common to decompose every time - `halves` or `split` - appears in 15+ rules - `has_AP` - arithmetic progressions are complex

**Drop specialized primitives that are easy to compose:** - `count_suit`, `count_color` → `length(filter(eq(get_suit, X)))` - `has_suit`, `has_color` → `any(eq(get_suit, X))` - `n_unique_X` → `unique_count(map(get_X))`

**Use standardized vocabulary for consistency:** - Makes language-primitive mapping more learnable - Enables systematic data augmentation - Creates cleaner training signal

---

## Part 5: Translation Model Requirements

### 5.1 What the Translation Model Must Learn

With standardized vocabulary, the model learns:

```
P("UNIFORM" | uniform) = high
P("COLOR" | get_color) = high
P("SUIT" | get_suit) = high
P("LEFT-HALF" | take, div, length) = high ← harder: 1 word → 3 primitives
```

### 5.2 Challenge: Multi-Primitive Phrases

When higher-level primitives are dropped, single phrases must map to multiple primitives:

| Phrase        | Primitives Needed                                                        |
|---------------|--------------------------------------------------------------------------|
| "first half"  | <code>take</code> , <code>div</code> , <code>length</code> , 2           |
| "same color"  | <code>unique_count</code> , <code>eq</code> , <code>get_color</code> , 1 |
| "has a heart" | <code>any</code> , <code>eq</code> , <code>get_suit</code> , HEARTS      |

This is **harder to learn** because: 1. IBM Model 4 assumes roughly 1:1 word-token alignment 2. Multi-primitive mappings require learning complex co-occurrences 3. Order of primitives matters but word order may not match

### 5.3 Mitigation Strategies

- Keep compound terms in vocabulary:**
  - "LEFT-HALF" as single token (not "LEFT HALF")
  - "SAME-COLOR" as single token
- Use neural encoder instead of IBM model:**
  - RNN/Transformer can learn multi-word → multi-primitive mappings
  - Attention mechanisms can align phrases to primitive sets
- Hierarchical descriptions:**
  - Top level: "LEFT-HALF COLOR UNIFORM"
  - Expanded: "take(div(length,2)) get\_color unique\_count eq 1"

## Summary

### Key Findings

1. **Current descriptions are high-level** and map directly to higher-level primitives
2. **Dropping higher-level primitives** increases program length  $\sim 2\times$  and search difficulty  $\sim 1000\times$
3. **Language CAN partially compensate** by biasing toward correct primitive combinations
4. **Standardized vocabulary helps** but requires learning multi-primitive mappings
5. **Hybrid approach recommended:** Keep critical higher-level primitives, standardize vocabulary

### Proposed Next Steps

1. **Create standardized vocabulary** (30-40 terms)
2. **Generate descriptions** in standardized format for all 45 rules
3. **Implement language encoder** (start with simple GRU, then BERT)
4. **Test with and without** higher-level primitives to measure language compensation
5. **Measure search effort reduction** attributable to language guidance

---

*Generated as part of /research-ultrathink analysis December 2024*